Neifit

Sabores tradicionales.
Una experiencia digital cercana.



Tienda virtual

Postres · Snacks · Bebidas

Integrantes

Maribel Marín Cortés
Nixon Garcia
Flor Neifix González

Una venta necesita más que un mensaje.

El problema que busca resolver Neifit

01

Información dispersa

Solicitudes, datos del cliente y productos en registros separados.

02

Poco control

Dificultad para conocer cantidades, disponibilidad y solicitudes pendientes.

03

Seguimiento limitado

El cliente necesita saber qué ocurrió con su pedido y su entrega.

Centralizar la información para atender,

De conocer la marca a recibir el pedido.

Una plataforma web con acceso según la función de cada persona

01

01 · Explorar

Landing institucional y muestra de productos.

02

02 · Ingresar

Registro e inicio de sesión con Supabase Auth.

03

03 · Solicitar

Carrito, pedido o cotización según la necesidad.

04

04 · Gestionar

Administración y seguimiento de la entrega.

React + Vite → FastAPI → Supabase / PostgreSQL: arquitectura objetivo

Una plataforma. Responsabilidades distintas.

¿Quiénes la utilizarían?

01

Cliente

Consulta productos, solicita cotizaciones, crea pedidos y consulta los propios.

02

Administra

Gestiona productos, solicitudes, usuarios, roles y permisos.

03

Mensajero

Consulta entregas asignadas y registra avances o novedades.

¿Qué necesita conocer el sistema?

La información se organiza en cuatro grupos

Personas y acceso 01

Nombre, correo, teléfono, rol y permisos.

Catálogo 02

Categorías, productos, precios, ingredientes, alérgenos y stock.

03 Operación

Carritos, pedidos, cotizaciones, detalles y direcciones.

Control 04

Seguimiento, transacciones de pago y auditoría de accesos.

La base del modelo.

Entidades principales del catálogo y de los usuarios

Entidad	Atributos principales	Función
profiles	id, full_name, email, phone, role	Identidad operativa del usuario
categories	id, name, slug, is_active	Agrupar productos
products	id, category_id, name, price, stock, image_url	Describir la oferta y su disponibilidad
carts / cart_items	user_id, status / product_id, quantity	Organizar la intención de compra

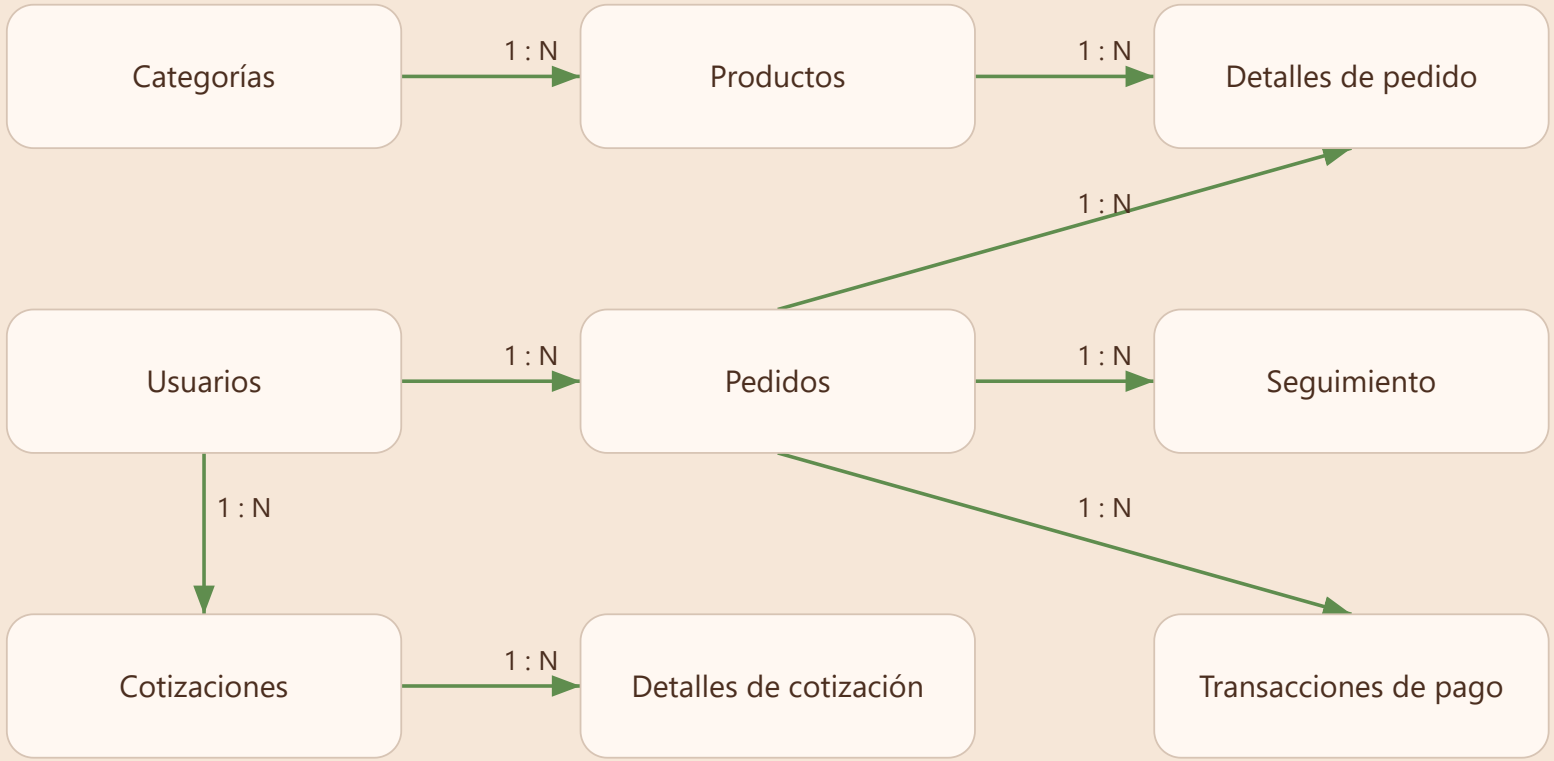
Cada solicitud conserva su detalle.

Entidades transaccionales del modelo diseñado

Entidad	Atributos principales	Función
orders	id, user_id, order_number, address, order_status, total	Registrar una compra
order_items	order_id, product_id, product_name, quantity, unit_price	Conservar los productos y precios del pedido
quotes / quote_items	quote_number, user_id, status / quantity, reference_price	Registrar una solicitud de cotización

Las relaciones dan coherencia al negocio.

Esquema conceptual simplificado · 1 = uno · N = muchos



El acceso depende de lo autorizado.

Permisos con nombres concretos para cada responsabilidad

Módulo o acción	Permiso	Alcance
Catálogo	catalog.read	Consultar productos
Pedidos propios	orders.read_own	Solo registros del cliente
Entregas asignadas	delivery.read_assigned	Solo pedidos asignados
Productos e inventario	products.manage	Gestión autorizada
Usuarios / Roles	users.manage / access.manage	Reservados al administrador

Reglas que protegen la información.

Integridad de datos y autorización

Datos válidos 01

Precios y cantidades positivos. Stock no negativo. Estados permitidos.

02 Referencias consistentes

Claves primarias y foráneas. Números de pedido y cotización únicos.

Compras coherentes 03

Un carrito activo por usuario. Un producto por

Acceso controlado 04

RLS en tablas privadas. Sin cambio del rol

Un modelo relacional para crecer con orden.

¿Por qué se diseñó de esta manera?

01

Separar responsa

Usuarios, productos, documento y detalles tienen funciones distintas.

02

Conserva el contexto

El precio del pedido no cambia cuando cambia el

03

Mantener trazabilidad

Cada avance de entrega y cambio de acceso puede quedar

Productos con pocas unidades

Identificar productos activos con cinco unidades o menos.

Reposición

Permite priorizar la revisión del inventario. Incluye productos agotados.

Datos requeridos

Tablas instaladas y registros

SQL / POSTGRESQL

```
select name, stock, price
from public.products
where is_active = true
  and stock <= 5
order by stock asc, name;
```

Cotizaciones pendientes

Consultar las solicitudes sin responder y su importe de referencia.

Atención comercial

El importe suma cantidad × precio de referencia; no incluye domicilio ni es un pago confirmado.

Datos requeridos

Tablas instaladas y registros persistidos en

SQL / POSTGRESQL

```
select q.quote_number,
q.customer_name,
q.created_at,

coalesce(sum(qi.quantity
* qi.reference_price), 0)
    as total_referencia
from public.quotes q
left join
public.quote_items qi on
qi.quote_id = q.id
where q.status =
'pendiente'
group by q.id,
q.quote_number,
q.customer_name,
q.created_at
order by q.created_at;
```

Entregas por mensajero

Consultar pedidos asignados que aún no están entregados ni cancelados.

Logística

Muestra responsable, cliente, dirección y estado. El JOIN excluye pedidos sin mensajero asignado.

Datos requeridos

Tablas instaladas y registros persistidos en PostgreSQL.

SQL / POSTGRESQL

```
select p.full_name as
mensajero, o.order_number,
  o.customer_name,
o.address, o.order_status
from public.orders o
join public.profiles p
  on p.id =
o.assigned_delivery_user_id
where o.order_status not in
('entregado', 'cancelado')
order by p.full_name,
o.created_at;
```

Productos con más unidades vendidas

Sumar unidades e importes de pedidos no cancelados con pago aprobado.

Análisis comercial

Agrupa por producto. El importe corresponde a sus detalles, sin sumar el domicilio.

Datos requeridos

Tablas

SQL / POSTGRESQL

```
select oi.product_id,
p.name,
  sum(oi.quantity) as
unidades,
  sum(oi.subtotal) as
importe_productos
from
public.order_items oi
join public.orders o
on o.id = oi.order_id
join public.products
p on p.id =
oi.product_id
where
o.payment_status =
'aprobado'
  and o.order_status
```

Usuarios y sus roles

Relacionar las cuentas registradas con el nombre de su rol.

Administración de accesos

Permite revisar asignaciones. Requiere la tabla access_roles y una cuenta autorizada.

Datos requeridos

Tablas instaladas y registros persistidos

SQL / POSTGRESQL

```
select p.full_name,
p.email, r.name as rol
from public.profiles p
join public.access_roles
r on r.code = p.role
order by r.name,
p.full_name;
```

Prototipo funcional, con una ruta de evolución.

Diferenciar lo implementado de lo pendiente

01

Implementado en el código

Interfaz y módulos comerciales. Autenticación con Supabase. Controles de acceso en la

02

Pendiente para producción

Persistir pedidos y cotizaciones en Supabase. Validar stock y totales con transacciones. Integrar y

Organizar la venta. Acompañar cada entrega.

Neifit conecta clientes, administración y mensajería en una misma experiencia.

1 · Demostrar

Recorrer catálogo, cotización y panel administrativo

2 · Consolidar

Conectar los documentos comerciales a PostgreSQL.

3 · Validar

Comprobar permisos, inventario y pagos de extremo a extremo.